Koen Essers • Master Internship • April 28, 2026

Notes Week 13

a. Poster

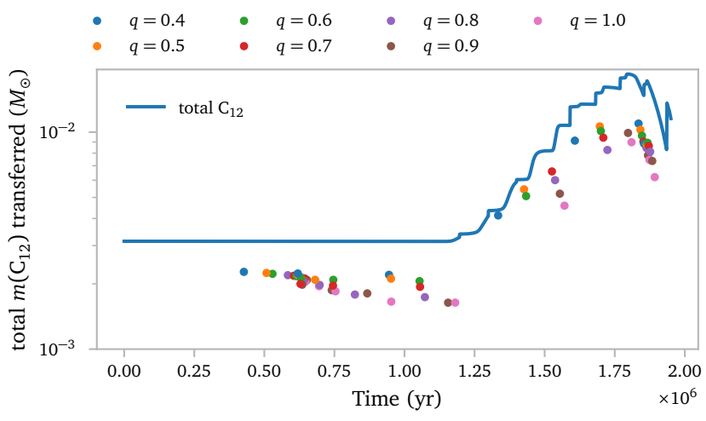
I spent the majority of this week on designing the poster for the NAC.

b. Mistake

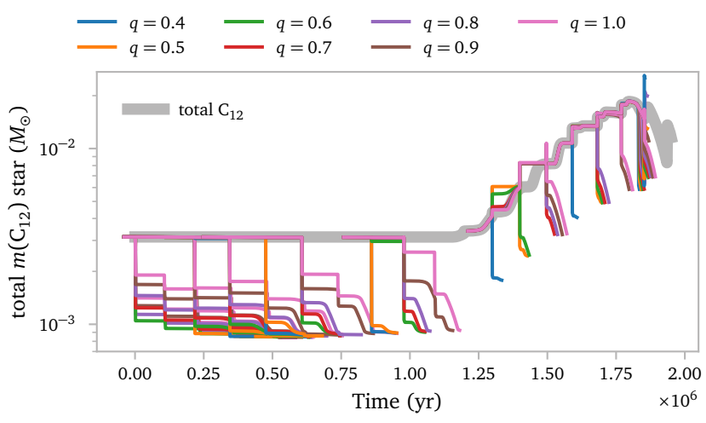
Even though I looked at the corrigendum from Temmink et al. (2025), and I explicitly set `use_radiation_corrected_transfer_rate` in my models, I very stupidly set it to `.true.` instead of `.false.`. This is fixed from now on.

c. Carbon transfer

I am interested in seeing how much carbon gets transferred to the accretor.



This plot shows the mass of C_{12} that is transferred at the end of the evolution of the binary. I compare it with the total amount of C_{12} available in the star as a function of time. The binary models all transfer a significant part of the available C_{12} to the companion, but not the total amount. This must be due to wind mass loss, and also for these models, the radiation corrected transfer.



This plot shows the amount of C_{12} that is left in the star for the various models. It clearly drops significantly when mass transfer occurs, which is what we would expect.

By assuming the carbon enriched mass that gets accretor onto the accretor is effectively mixed before the observations, we can quite easily compute the carbon abundance of the accretor after mass transfer.

We use that

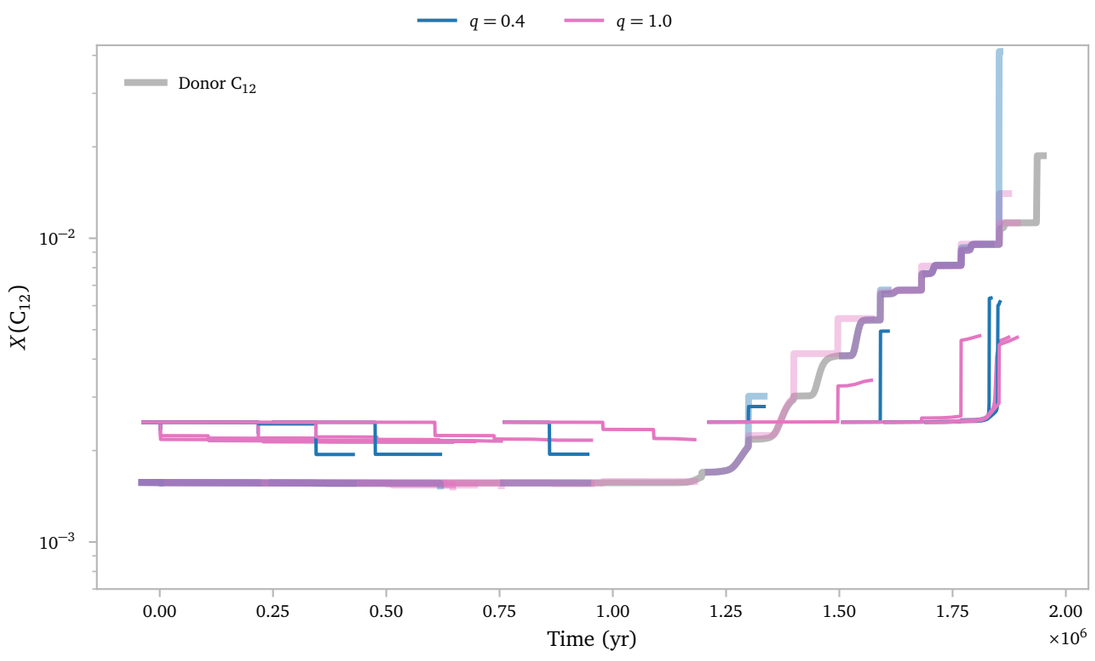
$$m(C_{12})_{\text{final}} = m(C_{12})_{\text{initial}} + m(C_{12})_{\text{transferred}}$$

$$X(C_{12}) = \frac{m(C_{12})_{\text{final}}}{M_{\text{star}}}$$

I also assume that the mass fraction of C_{12} in the accretor star is constant and does not change with the mass of the initial accretor[1].

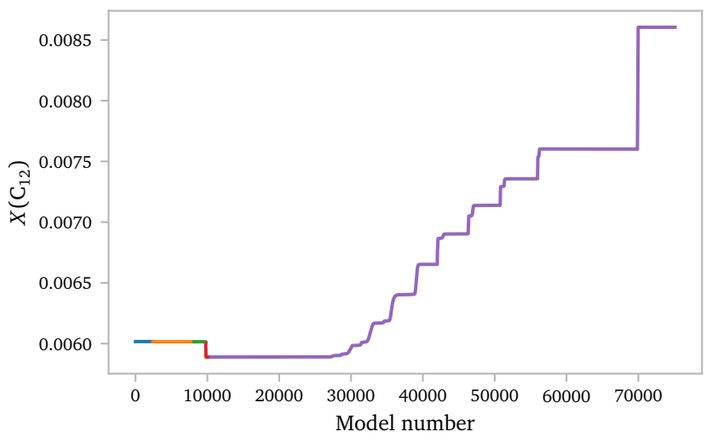
The plot below shows the approximate carbon 12 abundance of the accretor during the evolution.

1 Such that $X(C_{12})_{\text{heavy star}} = X(C_{12})_{\text{light star}}$



For models that interact early on during the TPAGB, the abundance drops, but once the carbon abundance in the TPAGB star exceeds the abundance of the accretor, the carbon abundance of the accretor will grow after mass transfer. The thick lines show the carbon abundance of the donor.

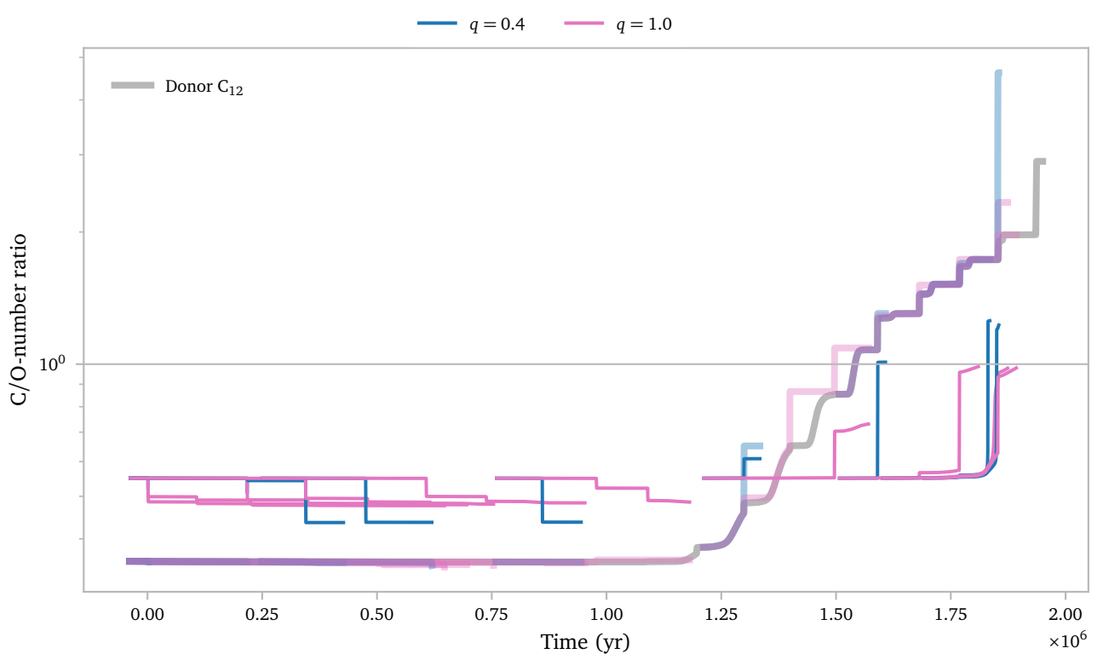
To compute the C/O-number ratio, we need to compute the mass ratio of O_{16} as well. Unfortunately, this element changes in abundance during the TPAGB:



The O_{16} mass fraction evolves quite similar to that of C_{12}, just a little less extreme, which causes the CO-number ratio to shift for single TPAGB stars.

This means we cannot assume it to remain constant and we must use the same procedure to compute the mass fraction of O_{16} for the accretor.

If we do this, and divide the two and multiply by the numerical factor 16/12 to turn the faction into a number ratio, we obtain:



Under these assumptions it seems like only if a star interacts quite late in the evolution, and with a significantly low mass ratio, it can appear as a carbon enhanced star. The assumptions are on one hand quite conservative. We assume the star mixes the accreted material throughout the *whole* star, which significantly reduces the effect of the accretion on the surface abundance compared to assuming no mixing. On the other hand, we assume [2] all of the mass is accreted, which is very probably an overestimation of the amount of total transferred mass.

2 nearly, because of the radiation losses

d. Circumbinary toroid

I looked through MESA's source code for its implementation of the circumbinary coplanar toroid. It is based on the same 2 parameters as Soberman et al. (1997), where δ determines how much of the *transferred* mass is lost to a toroid. Specifically, they define `mdot_system_cct = b% mtransfer_rate * b% mass_transfer_delta`. This is set as follows:

```
! $MESA_DIR/binary/private/binary_private_def.f90
! subroutine adjust_mdots

! mdot_system_transfer is mass lost from the vicinity of
 ↪  each star
! due to inefficient rlof mass transfer, mdot_system_cct
 ↪  is mass lost
! from a circumbinary coplanor toroid.
if (b% mtransfer_rate*b% mdot_wind_transfer(b% d_i) >= 0
 ↪  .or. b% CE_flag) then
    b% mdot_system_transfer(b% d_i) = 0d0
    b% mdot_system_transfer(b% a_i) = 0d0
    b% mdot_system_cct = 0d0
else
    b% mdot_system_transfer(b% d_i) = b% mtransfer_rate *
     ↪  b% mass_transfer_alpha
    b% mdot_system_cct = b% mtransfer_rate * b%
     ↪  mass_transfer_delta
    if (b% point_mass_i == 0 .or. b% model_twins_flag)
     ↪  then
        b% mdot_system_transfer(b% a_i) = b%
         ↪  mtransfer_rate * b% mass_transfer_beta
    else
        ! do not compute mass lost from the vicinity
         ↪  using just mass_transfer_beta, as
        ! mass transfer can be stopped also by going past
         ↪  the Eddington limit
        b% mdot_system_transfer(b% a_i) =
         ↪  (actual_mtransfer_rate + b% component_mdot(b%
         ↪  a_i)) &
            + b% mtransfer_rate * b% mass_transfer_beta
    end if
end if
```

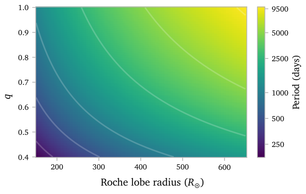
This is then used to compute the change in angular momentum in the default routine as:

```
! $MESA_DIR/binary/private/binary_jdot.f90
! subroutine default_jdot_ml

!mass lost from circumbinary coplanar toroid
b% jdot_ml = b% jdot_ml + b% mdot_system_cct * b% mass_transfer_gamma * &
     sqrt(standard_cgrav * (b% m(1) + b% m(2)) * b% separation)
```

I think just copying this line into my `wind_loss` subroutine should be all that is necessary to simulate mass loss from a coplanar circumbinary toroid. The new `wind_loss` subroutine is now:

```
! $RUN_DIR/src/bin/additional_routines.inc

subroutine wind_loss(binary_id, ierr)
    integer, intent(in) :: binary_id
    integer, intent(out) :: ierr
    type(binary_info), pointer :: b
    real(dp) :: q, vw_over_vorb
    real(dp) :: beta, eta
    real(dp) :: omega, I_eff
    real(dp) :: domega_dt_wind

    ierr = 0
    call binary_ptr(binary_id, b, ierr)
    if (ierr /= 0) then
        write (*, *) 'failed in binary_ptr'
        return
    end if

    ! --- compute q == m_accretor / m_donor and v
    q = b%m(b%a_i)/b%m(b%d_i)
    vw_over_vorb =
     ↪  compute_vwind(b%s_donor)/compute_vorbit(b%m(1) +
     ↪  b%m(2), b%separation)
    b%s_donor%xtra(x_vw_over_vorb) = vw_over_vorb

    ! --- get beta and eta from Saladino
    beta = compute_beta_Sal(q, vw_over_vorb)/sqrt(1 -
     ↪  pow2(b%eccentricity))
    b%s_donor%xtra(x_beta) = beta
    eta = compute_eta_Sal(q, vw_over_vorb)
    b%s_donor%xtra(x_eta) = eta

    ! --- star angular momentum loss due to winds
    ! compute the change in spin frequency. this
    ! gets used in extra_binary_finish_step.
    omega = b%s_donor%xtra(x_omega)
    I_eff = b%s_donor%xtra(x_I_eff)
    domega_dt_wind = (b%mdot_system_wind(b%d_i) / (1 -
     ↪  beta)
     ↪  *(b%s_donor%photosphere_r*Rsun)**2*omega/1.5)/I_eff
    b%s_donor%xtra(x_domega_dt_wind) = domega_dt_wind

    ! --- mass lost from vicinity of donor
    b%jdot_ml = b%mdot_system_wind(b%d_i) &
                *eta &
                *(b%separation)**2 &
                *sqrt(1 - pow2(b%eccentricity)) &
                *2*pi/b%period

    ! --- mass lost from circumbinary coplanar toroid
    b% jdot_ml = b% jdot_ml + b% mdot_system_cct * b%
     ↪  mass_transfer_gamma * &
                  sqrt(standard_cgrav * (b% m(1) + b%
                   ↪  m(2)) * b% separation)

end subroutine wind_loss
```

I simulate an analytic grid again using the same code I used last week to find a nice balance for predicted orbital periods on the MESA grid parameter space. I find that $\delta = 0.2$, $\gamma = 1.25$ gives the following set of predicted orbits:

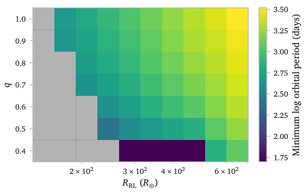


These values give a very broad region in the parameter space that generates periods between 1000 and 5000 days, with a few smaller and larger periods. I think this aligns well with the observed period distribution of barium stars as well, and hopefully should not be *too* challenging for mesa, since we are not trying to create extremely short periods.

e. Grid

I ran a grid with the same starting conditions as before, but now using $\delta = 0.2$ and $\gamma = 1.25$.

I start by showing the minimum orbit that is achieved during the binary evolution, because this shows a peculiar edge case that I had to account for.



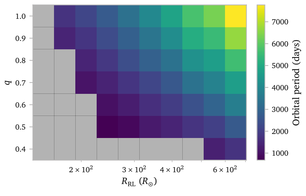
Two things should be noted here.

1. The are significantly more systems that *fail* due to a MESA error, even for large q.
2. There are some systems that reach extremely small periods.

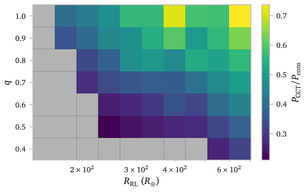
MESA should prevent systems from evolving past a certain point to stop spending resources on hopeless models. I actually implemented this in the grid already, which is why all models that experience this *issue* have the same minimum period of 50 days.

I prevented further evolution by implementing a very simple stopping condition in the `extra_binary_finish_step`.

From now on, I quantify these models as unstable as well. The period distribution of the new models looks like this:

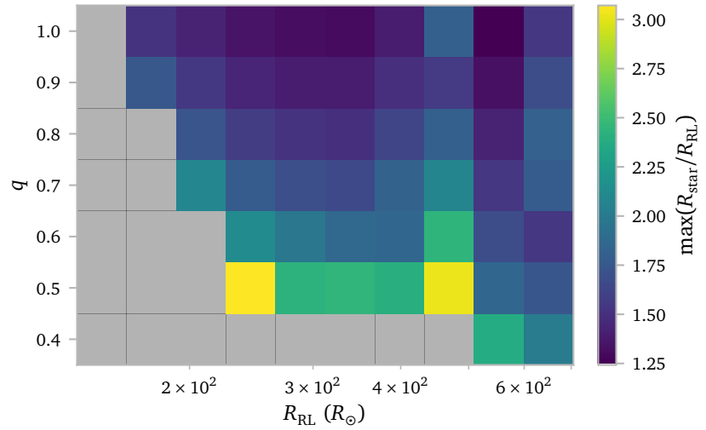


The periods that we find using with mass loss through the circumbinary ring are *significantly* lower than the periods we find without it. This is obviously what we would expect, but it is nice to see that MESA can, in many cases resolve the RLOF mass transfer, even with this extra mass loss.

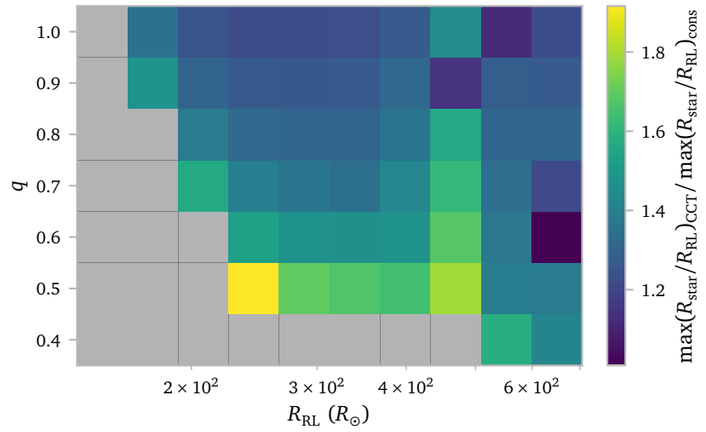


This figure compares the orbits from the previous grid with the orbits from the new grid. Clearly, in many cases, the orbits are a lot smaller.

It is also interesting to look at the maximum ratio of the star radius to the Roche lobe radius.

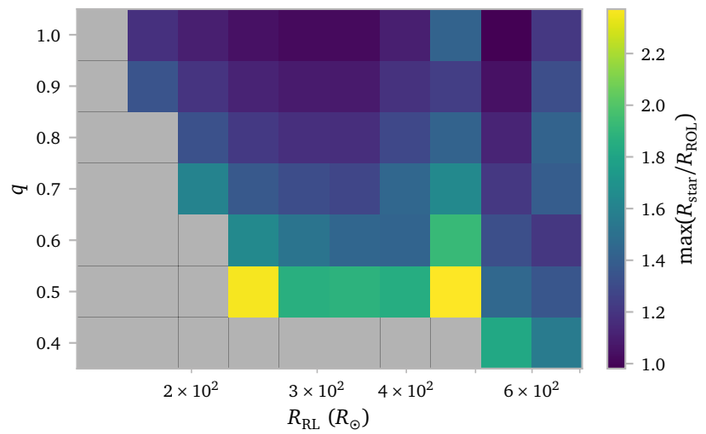


We see two systems with very high ratios, the other systems look more or less like the previous grid. It is easier to again compare the models directly.



Now we see that all models extend past their Roche lobe a bit more when mass is lost from the CCT.

Now, let's look at the outer Roche lobe radius:



All models reach their outer Roche lobe. This is a bit worrying, but it is also how mass is supposed to escape to the CCT.

References

Soberman, G. E., Phinney, E. S., & van den Heuvel, E. P. J. (1997) *Stability criteria for mass transfer in binary stellar evolution.*. A&A327, 620.

Temmink, K. D., Pols, O. R., Justham, S., Istrate, A. G., & Toonen, S. (2025) *Coping with loss: Stability of mass transfer from post-main-sequence donor stars (Corrigendum)*. A&A694, C8.